

# Your GEP interview prep

Ojas Srivastava · SVNIT Surat · Intern – Technology

Every question here was actually asked in a public candidate report. Nothing invented.

## 1. Your locked intro — say this, then stop

Good afternoon sir, ma'am. I am Ojas Srivastava. I am a pre-final year B.Tech student in Artificial Intelligence at SVNIT Surat, with a CGPA of 9.20.

Coursework: OOP, DBMS, OS, CN, DAA.

Google Solution Challenge 2026 Global rank 106 — LogiFlow, supply-chain routing. Vibe2Ship 2026 Global Top 20 — Community Hero, civic issue reporting PWA.

Coding workflow: Cursor, MCP and plugins. Spec and review every merge.

Chairperson Nexus SVNIT, CSE+AI departmental cell, manage and mentor 500+ students.

I own the outcome. GEP works on AI-powered procurement and supply chain. That is the kind of work I have already started on — taking operational data to a decision, and owning it till it ships.

**Do NOT volunteer in R1:** location, intern title, IFFCO (unless they open the resume), MCP server names, LangGraph, Azure, or “I led the whole team.” AirHelp is not on the resume — never bring it up.

## 2. Round 1 — minute map

- **0–3 min:** the locked intro. Then stop talking and let them pick.
- **3–13 min:** OOP. Pillars, then polymorphism and inheritance with a **real-world** example. Expect this.
- **13–25 min:** one project deep dive — almost always LogiFlow. Have the architecture and one hard bug ready.
- **25–40 min:** one Easy–Medium DSA. Strings or an array with a hashmap. Speak the approach before you type.
- **40–45 min:** “difficulties you faced,” then your questions for them.

## 3. Round 2 — minute map

- It may be **10 minutes or 40**. Answer completely on the first pass; assume no second chance.
- Opens with a fundamentals volley: purpose of OOP, DS vs algorithms, favourite data structure.
- **Say graph** when asked your favourite — it maps to the BFS question they ask next. Then stay there.
- One DSA, often with “**now do it another way**.” Fibonacci two ways is quoted verbatim in a real report.
- A puzzle: divide and conquer, 12 balls, or two buckets.

## 4. Intro and OOP

### Q. Tell me about yourself.

*Typical: Rambles through school marks and every course taken.*

**You say.** The locked intro on page 1, verbatim. Then stop.

### Q. Explain the pillars of OOP — with real-world examples.

*Typical: “Encapsulation, abstraction, inheritance, polymorphism” and nothing else.*

**You say.** Encapsulation — a route service hides Redis; callers only see `getRoutes()`. Abstraction — a car’s steering wheel; you turn it without knowing the rack. Inheritance — Vehicle → Car and Truck share start/stop, each adds its own behaviour. Polymorphism — I call `cost()` and a railway route and a road route each compute it differently.

### Q. Go deeper on polymorphism and inheritance.

*Typical: Repeats the definition louder.*

**You say.** Inheritance is “is-a” reuse; polymorphism is one interface, many behaviours. Compile-time is overloading; runtime is overriding through a base type. In LogiFlow three pipelines — railway, comparator, hybrid — all feed the same ranker. The ranker does not care which pipeline produced the corridor; it only ranks on time, cost and delay.

#### Q. What is the purpose of OOP?

*Typical: “To make code reusable.”*

**You say.** To model a system as objects that own their own data and behaviour. It keeps change local — I can swap the railway pipeline without touching the ranker. Reuse is a side effect, not the goal.

#### Q. Abstraction vs abstract class vs encapsulation.

*Typical: Mixes up abstraction and encapsulation.*

**You say.** Abstraction is a design idea — show what it does, hide how. An abstract class is the language tool — it cannot be instantiated and can hold unimplemented methods. Encapsulation is bundling data with the methods that touch it and restricting direct access. Abstraction hides complexity; encapsulation hides data.

#### Q. Access specifiers.

*Typical: “Public, private, protected” with no scope detail.*

**You say.** Public is visible everywhere, private only inside the class, protected inside the class and its subclasses. In Java, default is package-private. I keep fields private and expose behaviour, not state.

#### Q. What does static mean?

*Typical: “It means constant.”*

**You say.** Static belongs to the class, not to any object — one copy shared by all instances. A static method can be called without creating an object, so it cannot touch instance fields. I use it for helpers and counters.

#### Q. What is a friend function? (C++)

*Typical: Silence.*

**You say.** A friend function is declared inside a class but is not a member, and it is allowed to access private and protected data. It is used for things like operator overloading across two classes. It breaks encapsulation deliberately, so it should be rare. C++ has it; Java does not.

#### Q. Why is main public? Why String[] args?

*Typical: “Because Java needs it.”*

**You say.** Public because the JVM calls it from outside the class, so it must be visible. Static so it can run without creating an object. `String[] args` carries the command-line arguments, and it is an array because there can be many.

#### Q. Difference between data structures and algorithms.

*Typical: “Both are coding topics.”*

**You say.** A data structure is how data is stored and organised — an array, a hashmap, a graph. An algorithm is the sequence of steps that operates on it. The structure decides what the algorithm can do cheaply: a hashmap makes lookup  $O(1)$ , so my array problem becomes one pass instead of two loops.

#### Q. Stack vs queue vs list vs tree.

*Typical: Only defines stack and stops.*

**You say.** Stack is LIFO — undo, function calls. Queue is FIFO — BFS, job scheduling. A list is linear ordered access; array is contiguous with  $O(1)$  index, linked list is  $O(1)$  insert. A tree is hierarchical, and a balanced one gives  $O(\log n)$  search.

**Q. Python vs Java.**

*Typical: “Python is easier.”*

**You say.** Python is dynamically typed and interpreted — faster to write, which is why my FastAPI service is Python. Java is statically typed and compiled to bytecode, so the compiler catches type errors and it runs faster. Java for large typed systems, Python for data and ML work.

**Q. MySQL vs MongoDB.**

*Typical: Fakes Mongo experience.*

**You say.** MySQL is relational — fixed schema, joins, strong ACID guarantees. MongoDB is document-based — flexible schema, scales horizontally, no joins. I have not used Mongo; the closest I have worked with is Firestore, which is also document-based. My relational work is MySQL at IFFCO.

**Q. Primary key vs foreign key.**

*Typical: “Primary key is the ID.”*

**You say.** A primary key uniquely identifies a row — unique, not null, one per table. A foreign key points to a primary key in another table and enforces referential integrity. In a routing schema, corridor id would be the PK; a route-leg table would FK to it so you cannot store a leg for a corridor that does not exist.

**Q. What is your favourite data structure?**

*Typical: Says “tree,” then cannot explain rotations.*

**You say.** Graph. My whole routing problem is a graph — stations as nodes, corridors as edges weighted by time, cost and delay. I use adjacency lists because the graph is sparse, and BFS when every hop counts equally. Dijkstra when the edges carry real weights.

**Q. How would you know what language a website is built in without reading its code?**

*Typical: “You cannot.”*

**You say.** Check the response headers — `Server` and `X-Powered-By` often leak it. File extensions and URL patterns give it away, like `.php` or Next.js's `_next` paths. Cookie names too — `JSESSIONID` means Java, `PHPSESSID` means PHP. Or the error page style.

**Q. Design a bank account system using OOP.**

*Typical: Writes one giant class.*

**You say.** An abstract `Account` with `balance`, `deposit()` and `withdraw()`. `Savings` and `Current` extend it and override `withdraw()` for different overdraft rules — that is polymorphism. `Balance` stays private, changed only through methods, so I can validate and log every change. A `Transaction` class records each operation.

**Q. Design Zoomcar / a car rental system.**

*Typical: Jumps straight to database columns.*

**You say.** Core entities: `User`, `Car`, `Location`, `Booking`, `Payment`. A `Booking` links user, car and a time range, and the key constraint is no two bookings overlapping for the same car. Search is by location and time window, so I index on car ID plus start time. For the double-booking case I take a lock or use a unique constraint on the slot.

**Q. Design a timetable database.**

*Typical: One table with everything in it.*

**You say.** Tables: `Teacher`, `Subject`, `Room`, `Batch`, and a `Slot` table with `day` and `period`. The timetable row references teacher, subject, room, batch and slot. The uniqueness rules do the work — one teacher cannot be in two rooms in the same slot, and one room cannot hold two classes.

**Q. Two users grab the last item in a cart at the same time. What happens?**

*Typical: “The second one gets an error.”*

**You say.** That is a race condition — both read stock as 1 and both write 0. The fix is to make check-and-decrement atomic: a transaction with a row lock, or a conditional update like `SET stock = stock - 1 WHERE stock > 0` and check rows affected. Whoever loses gets “out of stock.” In LogiFlow I hit the same idea with Redis — SQL stays the source of truth, the cache never decides.

## 5. Projects — they ask only what is on your resume

### Q. Explain the key projects you have implemented.

*Typical:* Lists six projects with no depth on any.

**You say.** Two. LogiFlow — supply-chain route optimisation, Google Solution Challenge global rank 106, I was technical co-lead. Community Hero — a civic issue reporting PWA I built solo, Vibe2Ship global top 20. I would start with LogiFlow since it is closest to what GEP does.

### Q. Walk me through LogiFlow.

*Typical:* “It uses AI to optimise logistics.”

**You say.** It ranks freight routes on time, cost and delay so a planner can pick one. Three pipelines — railway, comparator and hybrid — feed one ranker, built on 15,000-plus train-days across 580-plus corridors. Stack is FastAPI, Next.js and Redis, responses land in 100 to 400 milliseconds. Test suite is 100 of 100 passing on pytest.

### Q. What difficulties did you face in the project?

*Typical:* “Time management was hard.”

**You say.** Latency. The first version recomputed rankings on every request and took seconds. I moved to cache-aside with Redis — check the cache, on a miss compute and store with a TTL, and SQL stays the source of truth. That took it to 100–400 milliseconds.

### Q. Why Redis? What happens if the cache is stale?

*Typical:* “Redis is fast.”

**You say.** Redis is in-memory, so a lookup is sub-millisecond, and my ranking inputs do not change per request. I use cache-aside with a TTL, and I invalidate on write. If Redis is down the request still works — it just recomputes from SQL, which is always the source of truth.

### Q. Which algorithms did you actually use?

*Typical:* Name-drops “machine learning.”

**You say.** Graph search over the corridor network — BFS when hops are uniform, weighted shortest path when they are not. Then a scoring pass that ranks candidate routes on time, cost and delay. The heavy lifting is the graph plus the ranking, not a model.

### Q. Tell me about your AI project. (Community Hero)

*Typical:* “I called the Gemini API.”

**You say.** It is a civic issue reporting PWA — a citizen photographs a problem and it gets routed to the right department. I built it solo on React, Express, Firestore and Cloud Run. Gemini classifies the report into a category, with guardrails on the output. If the model is unsure or unavailable, it falls back to a manual admin queue.

### Q. How did you do login and authentication?

*Typical:* “I used Firebase login.”

**You say.** Firebase Auth issues a token on the client; my Express layer verifies it on every protected route before touching Firestore. Roles are checked server-side, so a citizen cannot hit an admin endpoint. I never trust the client for authorisation.

### Q. Tell me about your internship. (only if they open the resume)

*Typical: Vague on what was actually built.*

**You say.** IFFCO, June to July 2025. I built backend services on Node, Express and MySQL — 10-plus REST endpoints supporting 50-plus daily workflows. I containerised it with Docker and put it on a CI/CD pipeline. That is where I learned to write for someone else's team.

### Q. What is Career Automation? (only if they open the resume)

*Typical: Oversells it as a product.*

**You say.** A personal automation stack. One part reaches 966 companies and 10,700 contacts over rate-limited SMTP. The other is OA Forge — 14,000-plus practice questions with 5,500-plus tests, judged by a C++ g++ runner. The public GitHub repo is the YAML workflows only.

### Q. Tell me about Nexus. (only if they open the resume)

*Typical: "I lead 500 people."*

**You say.** Nexus is the CSE and AI departmental cell at SVNIT. I am Chairperson — I run the events and mentor the students who come through them; 500-plus is event reach, not daily headcount. I got there over about two years: Member, SMC, Academic Mentor, Tech Rep, then Chairperson.

### Q. Have you used MongoDB / C# / Angular / Hadoop?

*Typical: Bluffs, then gets caught two questions later.*

**You say.** Mongo — I have not used it; the closest is Firestore, which is also document-based. C# and Angular — I have not used them, but I can pick them up. Hadoop — I have not run a cluster; I know MapReduce is a map step then a reduce step over distributed data. I would rather say that than guess.

## 6. DSA — exactly what has been asked

### Q. First non-repeating character in a string.

*Typical: Nested loops,  $O(n^2)$ , no counting.*

**Method.** Pass 1 — count each character in a hashmap or a 256-size array. Pass 2 — walk the string left to right, return the first character whose count is 1. If none, return a sentinel.

**Dry run.** "swiss" → counts s=3, w=1, i=1. Scanning: s (3, skip), w (1) → answer w. Time  $O(n)$ , space  $O(1)$  for fixed alphabet.

**You say.** Two passes. Count characters in a map, then scan the string in order and return the first with count one. For "swiss" that gives w.  $O(n)$  time, constant space for a fixed alphabet.

### Q. Remove repeating characters and print the result.

*Typical: Uses a nested loop and mangles the order.*

**Method.** Keep a **seen** set. Walk the string once; if the character is not in **seen**, append it to the output and add it. Order is preserved because you never look backwards.

**Dry run.** "programming" → "progamin". Time  $O(n)$ , space  $O(k)$ .

**You say.** One pass with a seen set. Append a character only the first time I meet it, so the original order survives. "programming" becomes "progamin".  $O(n)$  time.

### Q. Find the number of seconds between two given times.

*Typical: Subtracts hours and minutes separately, gets negatives.*

**Method.** Convert each time to total seconds:  $h*3600 + m*60 + s$ . Subtract. If the result is negative, the second time is next day — add 86400.

**Dry run.** 23:50:00 → 00:10:00 gives  $600 - 85800 = -85200$ ; add 86400 → **1200** seconds.  $O(1)$ .

**You say.** Normalise both times to seconds since midnight, then subtract. If it comes out negative the second time crossed midnight, so I add 86400. Constant time, and it removes all the borrow bugs.

### Q. Array problem — solve it using a map.

*Typical: Two nested loops.*

**Method.** One pass, hashmap as memory. For pair-sum: for each  $x$ , look up  $\text{target} - x$  in the map; if present you have the pair, else store  $x$  with its index. Same trick for duplicates, frequencies and subarray sums.

Time  $O(n)$ , space  $O(n)$  — you are trading memory for the inner loop.

**You say.** The map replaces the inner loop. For two-sum I ask the map for target minus the current value before I insert the current value, so it is one pass.  $O(n)$  time,  $O(n)$  space instead of  $O(n^2)$ .

### Q. Maximum subarray sum. (Kadane — not quoted, but Easy array is confirmed)

*Typical: Tries every subarray.*

**Method.** Track  $\text{cur}$  and  $\text{best}$ . For each element:  $\text{cur} = \max(x, \text{cur} + x)$  — either extend the running subarray or restart at  $x$ . Then  $\text{best} = \max(\text{best}, \text{cur})$ .

**Dry run** on  $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$ :

$\text{cur}$ : -2, 1, -2, 4, 3, 5, 6, 1, 5     $\text{best}$ : -2, 1, 1, 4, 4, 5, 6, 6, 6     $\rightarrow$  answer 6 (the subarray  $[4, -1, 2, 1]$ ).  $O(n)$  time,  $O(1)$  space.

**You say.** Kadane. At each element I either extend the current subarray or start fresh at that element, whichever is larger, and I keep the best seen. On the standard array it gives 6, from 4, -1, 2, 1. One pass, constant space.

### Q. Print Fibonacci numbers in a given range — now do it a second way.

*Typical: Writes plain recursion and freezes when asked for another way.*

**Method. Method 1 — iterative.** Keep  $a=0$ ,  $b=1$ ; loop, print when the value falls in  $[L, R]$ , stop past  $R$ .  $O(n)$  time,  $O(1)$  space.

**Method 2 — recursion with memoisation.**  $\text{fib}(n)$  returns from a map if computed, else  $\text{fib}(n-1)+\text{fib}(n-2)$  and store.  $O(n)$  time,  $O(n)$  space. Plain recursion without the map is  $O(2^n)$  — say that out loud.

**You say.** Two ways. Iteratively I hold the last two numbers and roll forward, printing anything inside the range —  $O(n)$  time, constant space. Or recursion with a memo map, which is also  $O(n)$  but costs  $O(n)$  space for the stack and the table. Plain recursion is exponential, so I would not ship it.

### Q. Alt+Tab — move the k-th item to the front.

*Typical: Sorts the list.*

**Method.** Take the element at index  $k-1$ , remove it, insert it at index 0; the rest shift right by one. Guard first: if  $k > n$  or  $k \leq 0$ , either wrap with  $k = ((k-1) \% n) + 1$  or return the list unchanged — ask which they want.

**Dry run.**  $n=4$ ,  $\{1,2,3,4\}$ ,  $k=3 \rightarrow$  pull 3  $\rightarrow \{3,1,2,4\}$ . Time  $O(n)$ , space  $O(1)$  in place.

**You say.** Pull the  $k$ -th element out and put it at the front; everything before it shifts one right. For 1,2,3,4 with  $k$  equal to 3 that gives 3,1,2,4. I would confirm the edge case first — if  $k$  is at least  $n$ , do I wrap it with a modulo or leave the list as it is?

### Q. Find the middle of a linked list.

*Typical: Counts the length, then loops again.*

**Method.** Slow and fast pointers from head. Move slow one step, fast two. When fast or  $\text{fast.next}$  is null, slow is at the middle. For an even length this lands on the second middle — move fast's start back one if they want the first.

One pass,  $O(n)$  time,  $O(1)$  space.

**You say.** Two pointers — slow moves one node, fast moves two. When fast hits the end, slow is at the middle. One pass, no length count, constant space.

### Q. Write binary search.

*Typical: Writes  $(lo+hi)/2$  and an infinite loop.*

**Method.**  $lo = 0$ ,  $hi = n-1$ ; while  $lo \leq hi$ :  $mid = lo + (hi-lo)/2$ ; if  $a[mid] == x$  return  $mid$ ; if  $a[mid] < x$  then  $lo = mid+1$  else  $hi = mid-1$ ; return -1. Array must be sorted.

$O(\log n)$  time,  $O(1)$  space.  $lo + (hi-lo)/2$  avoids integer overflow.

**You say.** The array has to be sorted. I keep a low and high bound and check the midpoint each time, dropping the half that cannot hold the target. I write mid as low plus half the gap to avoid overflow.  $O(\log n)$ .

### Q. Print prime numbers from 1 to 100.

*Typical:* Checks divisibility up to  $n$  for every number.

**Method.** Sieve of Eratosthenes: mark an array of size 101 true, set 0 and 1 false. For  $i$  from 2 while  $i*i \leq 100$ , if  $i$  is still true mark every multiple from  $i*i$  as false. Print what remains.

$O(n \log \log n)$ . There are **25** primes below 100. If they want a single number check, trial-divide only up to  $\sqrt{n}$ .

**You say.** Sieve of Eratosthenes — start with everything marked prime, then cross out multiples of each prime starting from its square. Below 100 that leaves 25 primes. It is  $n \log \log n$ , much better than testing each number separately.

### Q. Time complexities of the sorting algorithms.

*Typical:* “All of them are  $n \log n$ .”

**Method.** Quick: average  $O(n \log n)$ , worst  $O(n^2)$ , in place, not stable. Merge:  $O(n \log n)$  always, stable,  $O(n)$  extra space. Heap:  $O(n \log n)$  always, in place, not stable. Insertion:  $O(n^2)$ , but  $O(n)$  on nearly sorted data. Bubble:  $O(n^2)$ .

**You say.** Merge and heap are  $n \log n$  in every case; quicksort is  $n \log n$  on average but  $n$  squared if the pivots are bad. Merge is stable and needs  $O(n)$  extra space, heap and quick are in place. Insertion is  $n$  squared but linear on nearly sorted input.

### Q. BFS — and its complexity.

*Typical:* Says BFS uses recursion.

**Method.** Queue plus a visited set. Push the source and mark it. While the queue is not empty: pop a node, and for each unvisited neighbour mark it and push it. Level order falls out naturally — process the queue one level’s worth at a time.

$O(V + E)$  time with an adjacency list,  $O(V)$  space. DFS uses a stack or recursion; BFS uses a queue.

**You say.** BFS uses a queue and a visited set — pop a node, push its unvisited neighbours, mark as you push. That gives level order and the shortest path in an unweighted graph.  $O(V \text{ plus } E)$  time,  $O(V)$  space with an adjacency list.

### Q. Chocolates — pick a range, boxes divisible by $M$ , maximise per box. (OA)

*Typical:* Brute-forces every range.

**Method.** Prefix sums so any range total is  $O(1)$ . The per-box value is the range total divided by  $M$ , so you want the maximum range total that  $M$  divides. Sweep ranges with the prefix array, or binary search the answer and check feasibility.

$O(n)$  prefix build;  $O(n \log n)$  with the binary search. First question to ask: is the range contiguous?

**You say.** Prefix sums first, so I get any range total in constant time. Then the per-box amount is the range total over  $M$ , so I am really maximising the range total that  $M$  divides cleanly. Before coding I would confirm whether the range has to be contiguous — that changes the whole approach.

## 7. Puzzles

### Q. A divide-and-conquer puzzle.

*Typical:* Starts guessing numbers.

**You say.** The pattern is always the same — split the input in half, solve each half, then combine. Merge sort splits, sorts and merges in  $n \log n$ . Binary search only needs one half each time, so it is  $\log n$ . I say the recurrence out loud and solve it.

### Q. Twelve balls, one is faulty. Find it.

*Typical:* Says four weighings.

**You say.** Three weighings on a balance. Split into three groups of four and weigh two of them — if they balance the odd ball is in the third group, otherwise it is on the heavier or lighter side. That leaves four, and I split those to find the group of one or two. Each weighing has three outcomes, so three weighings cover 27 cases and 12 fits.

**Q. Two buckets — measure an exact quantity.**

*Typical: Keeps pouring at random.*

**You say.** With a 5 and a 3 litre bucket to get 4: fill the 5, pour into the 3, and 2 litres are left. Empty the 3, move those 2 into it, refill the 5, then top up the 3 — it only takes 1 more. That leaves exactly 4 in the big bucket.

## 8. HR — the exact questions people were asked

**Q. What do you know about GEP?**

*Typical: “It is a good company in procurement.”*

**You say.** GEP works in procurement and supply chain across three arms — software, consulting and managed services. The software is SMART for source-to-pay, NEXXE for supply chain, with Quantum Intelligence as the AI layer across them. It is that mix of consulting judgement and shipped product that drew me.

**Q. How did you apply / how did you hear about us?**

*Typical: “My placement cell told me.”*

**You say.** Through the campus process at SVNIT, after the pre-placement talk. I read up on GEP afterwards and the procurement and supply-chain focus lined up with what I was already building.

**Q. Are you willing to relocate to Mumbai?**

*Typical: Hesitates, then negotiates.*

**You say.** Yes. I am open to both Mumbai and Hyderabad — whichever office the team is in.

**Q. What is your location preference?**

*Typical: “Somewhere near home.”*

**You say.** Both work for me. Mumbai or Hyderabad, no preference — I would rather be where the team is.

**Q. What are your hobbies / interests?**

*Typical: “Coding.”*

**You say.** I build things outside coursework — that is how LogiFlow and Community Hero happened. Outside that I run and I read. And I enjoy the mentoring side of Nexus more than I expected to.

**Q. Where do you see yourself in five years / what is your future goal?**

*Typical: “In a managerial position.”*

**You say.** Still building, but owning a system end to end rather than a feature. I want to be the person a team asks when something in production breaks. Long term, applied AI on real operational data — which is where GEP already works.

**Q. Tell me about your family.**

*Typical: Overshares for two minutes.*

**You say.** I am from a supportive family that has backed my choice to study AI. They are fine with me relocating for work. That has made it easy for me to focus on building things.

**Q. Tell me about your college.**

*Typical: Recites the ranking.*

**You say.** SVNIT Surat, an NIT. I am in the B.Tech AI programme, currently pre-final year with a 9.20 CGPA. Alongside coursework I chair Nexus, our CSE and AI departmental cell.

**Q. What is your favourite subject?**

*Typical:* “All of them.”

**You say.** DBMS, and OOP close behind. DBMS because indexing and transaction design showed up immediately in my own projects — the race-condition and caching decisions in LogiFlow came straight out of it.

**Q. Strengths and weaknesses.**

*Typical:* “My weakness is I work too hard.”

**You say.** Strength — I own outcomes; if something breaks in my project it is mine to fix, whoever wrote it. Weakness — I used to go too deep on polish before shipping. I fix it now by setting a scope and a date up front and reviewing every merge against it.

**Q. Why are you moving from AI to software engineering?**

*Typical:* “AI jobs are hard to get.”

**You say.** I am not moving away from it — the AI only matters once it is inside a working system. Most of my time on LogiFlow went into the API, the caching and the tests, not the model. I want to build the product that the intelligence sits inside, and GEP does exactly that.

**Q. What are your expectations / when can you join?**

*Typical:* Starts negotiating stipend.

**You say.** I am looking for real ownership — a piece of the system that is mine to ship. On timing I am fully available for the internship window and I can start whenever the team needs me.

## 9. Last check before you walk in

- Intro verbatim, then **stop**. Silence is theirs to fill.
- Every claim on the resume is a question. LogiFlow, Community Hero, IFFCO, Career Automation, Nexus — know the numbers.
- Favourite data structure → **graph**. Do not switch mid-answer.
- Not used it? Say so, then say the nearest thing you have used. Mongo → Firestore. Hadoop → map then reduce.
- Speak the approach and the complexity **before** you write code.
- “Now do it another way” is coming. Have the second method ready for Fibonacci especially.
- Location: yes to both Mumbai and Hyderabad. Answer it in one sentence.